

# Final Report: Vectorized Loop Shape Optimization in MLIR

MUHAMMAD MAZAYA, mmazaya

JUSTIN KIM, yutarkk

Modern SIMD architectures offer wide vector registers and predication support, but choosing optimal vector shapes and remainder-handling strategies remains challenging. Traditional compilers rely on fixed-width vectorization or simple heuristics that commit early to a single strategy, potentially missing better alternatives.

We present an MLIR pass that systematically explores vector shape configurations by generating multiple vectorized loop variants, each with distinct logical vector widths, remainder-handling strategies (scalar, masked, unrolled), and masking approaches. Rather than committing to a single strategy, our pass enumerates candidates and evaluates them using cost models before materializing the optimal choice.

We implement two complementary cost models: an analytical model based on ISA-specific masking and register pressure costs, and a neural-network model trained on synthetic and benchmark data. The analytical model uses a simple cost function calibrated via parameter sweeping, while the neural network captures non-linear microarchitectural interactions.

Our evaluation across synthetic matrix benchmarks, transformer blocks, and neural network workloads demonstrates substantial benefits: systematic strategy selection achieves up to 6.5x speedup over scalar remainder on AVX-512, with the analytical cost model consistently performing at or near the top across all ISAs and benchmarks. Results reveal strong ISA and workload dependencies: wide-vector ISAs (AVX2/AVX-512) benefit most from masking strategies, while real-world transformer workloads are often remainder-insensitive with modest 5–13% gains.

Our findings validate principled, analytically-guided cost modeling as a strong foundation for automatic vector-shape selection, while highlighting the challenges of neural-network-based models when they diverge from analytical predictions.

## 1 Introduction

Modern SIMD ISAs provide increasingly wide vector registers, predication, and masked memory operations, and compiler vectorizers must choose how to best exploit these features. Prior architectural guidance, for example, Intel’s Optimization Reference Manual, explicitly recommends using the maximum available SIMD width to achieve peak throughput [7]. In practice, production compilers such as LLVM rely on a combination of fixed-width defaults, target-specific heuristics, and user-supplied directives (e.g., pragmas to control vectorization width or enable/disable autovec) [2, 8].

Recent work from the LLVM and Arm communities has introduced more adaptive and architecture-aware heuristics that refine how vector width and remainder-handling strategies are chosen during lowering [9, 10]. However, even these advanced heuristics select a single vectorization shape early, which may be suboptimal when loop trip counts are irregular, data alignment is unknown, or the scalar remainder region dominates runtime.

In this project, we explore a more flexible approach within MLIR: instead of committing to one strategy, we generate multiple vectorized loop variants, each with a distinct “vector shape” (logical vector width, unroll factor, and remainder-handling strategy). A cost model evaluates these variants before lowering to LLVM IR, allowing the compiler to select the most efficient shape for the given loop and target ISA.

Our contributions are:

- A systematic enumeration of vectorization strategies (vector widths, unrolling, and masked remainder) as candidate variants.
- An analytical cost model that accounts for masking overhead and register pressure.

- A neural-network-based cost model, trained on synthetic and real benchmark data, to refine predictions where analytical simplifications are insufficient.

## 2 Background

### 2.1 Vectorization and Logical Vector Sizes

Loop-level vectorization transforms a scalar loop into operations on SIMD vectors by grouping iterations and replacing scalar operations with vector instructions. Rather than always using the full hardware vector width, compilers can choose a logical vector size (LS) that may be smaller than or equal to the hardware vector size (VS). Choosing  $LS < VS$  enables uniform masking throughout the loop, potentially eliminating remainder handling entirely when the iteration count divides evenly by LS. Additionally, loop unrolling can expose more SIMD parallelism by reducing loop-control overhead and increasing the workload per vector invocation [1, 5].

The choice of logical vector size and unrolling factor influences data alignment, register pressure, and instruction-cache behavior. On modern hardware, these secondary effects can dominate the benefits of wider vectors or more aggressive vectorization strategies.

### 2.2 Remainder Handling and Masked Execution

When the loop iteration count is not a multiple of the vector width, compilers must handle the remainder. Common strategies include scalar tails, loop peeling to align to vector boundaries, masked epilogues, or full-body masked execution for all iterations. Masking is appealing especially on ISAs with first-class vector predication (e.g., AVX-512, SVE), but masked operations carry latency and may increase register and dependency pressure [5].

### 2.3 Cost Models and Machine-Learning in Compiler Optimization

Traditionally, compilers use hand-crafted cost models to decide when and how to vectorize. With increasing microarchitectural complexity, these models become brittle. Recently, researchers have proposed learned cost models that predict the benefit of transformation sequences. For example, the work in the *Tiramisu* compiler uses a deep learning based cost model to predict speedups from transformations including tiling and unrolling [3]. Similarly, end-to-end vectorization using reinforcement learning has been explored under the name NeuroVectorizer [6].

### 2.4 MLIR and Architecture-Aware Lowering

The intermediate representation (IR) provided by MLIR simplifies the implementation of transformation passes, and the vector dialect supports explicit vector and masked operations. The work AXI4MLIR [4] recently demonstrated how MLIR can integrate architectural constraints (memory ports, vector width, alignment) into hardware-accelerator lowering pipelines. This inspires the design of our pass: we treat vector width, mask support, and alignment constraints as first-class configuration parameters in the compilation pipeline.

### 2.5 FMA Instruction

Modern SIMD architectures provide fused multiply-add (FMA) instructions that compute  $a \cdot b + c$  in a single operation with a single rounding step. FMA is critical for matrix multiplication performance because the inner product computation consists entirely of multiply-accumulate operations:

```
for (int k = 0; k < K; k++)
    C[i][j] += A[i][k] * B[k][j];
```

On AVX-512, the `vmadd231ps` instruction perform 16 simultaneous FMA operations in a single cycle, providing both higher throughput (2x vs. separate multiply and add) and improved numerical accuracy.

### 3 Designing the Vectorized Loop Shape Optimization Pass

#### 3.1 `linalg.matmul` as the Main Target

We implement the shape optimization pass design focusing on `linalg.matmul` because it provides a clean, self-contained setting for studying vector-shape decisions. Matmul is easy to benchmark, widely used across scientific and ML workloads, and already exercises the core problems we care about: choosing a VS, choosing an LS, and handling remainders when the iteration space is not divisible by either. These shape-divisibility issues fully appear in the  $N$ -dimension of matmul, so implementing a single pass for this one Linalg op is sufficient to expose the entire design space of remainder strategies and vector forms.

#### 3.2 From `linalg.matmul` to a Vectorized Loop

Our pass transforms high-level `linalg.matmul` operations into explicit vectorized loop nests using MLIR's structured control flow and vector dialects. The transformation decomposes the structured matmul into three nested loops: an outer loop over rows ( $i$ ), a middle loop over columns processed in vector-sized chunks ( $j$ ), and an inner reduction loop over the shared dimension ( $k$ ). For each iteration, the scalar element from the left-hand matrix  $A[i, k]$  is broadcast into a vector, a vector slice is loaded from the right-hand matrix  $B[k, j : j + \text{vectorLen} - 1]$ , and the result is accumulated using vectorized FMA operations. This maintains a vector accumulator across the  $k$ -dimension, avoiding scalar reductions that would break vectorization.

The transformation produces different code structures depending on whether the matrix dimensions are perfectly divisible by the vector length. Consider a  $4 \times 4$  matrix multiplication with a vector length of 4 elements (128-bit AVX for 32-bit floats). Since  $N = 4$  is perfectly divisible by the vector length, no remainder handling is needed.

If we have an MLIR `linalg.matmul`, it will be transformed into nested loops with a single vectorized path:

```
// Pseudocode (perfectly divisible case)
for i in 0 .. 4:
  for j in 0 .. 4 step 4:           // vector chunk
    acc = vector<float,4>(0)
    for k in 0 .. 4:
      a_scalar = A[i,k]
      a_vec = broadcast(a_scalar, 4)
      b_vec = load_vector(B[k, j : j+3])
      acc = fma(a_vec, b_vec, acc)
    store_vector(acc, C[i,j])
```

In this perfect-fit case, the remainder handling code is completely eliminated at compile time since  $N \bmod \text{vectorLen} = 0$ .

For rectangular matrices where  $N$  is not divisible by the vector length, the transformation generates an additional scalar cleanup loop. Consider a  $4 \times 6$  matrix with vector length 4, where  $N = 6$  and  $6 \bmod 4 = 2$ .

```
// Pseudocode (with scalar remainder)
for i in 0 .. 4:
  // Vectorized path for columns 0-3
```

```

148   for j in 0 .. 4 step 4:
149       acc_vec = vector<float,4>(0)
150       for k in 0 .. 8:
151           a_scalar = A[i,k]
152           a_vec = broadcast(a_scalar, 4)
153           b_vec = load_vector(B[k, j : j+3])
154           acc_vec = fma(a_vec, b_vec, acc_vec)
155       store_vector(acc_vec, C[i,j])
156
157   // Scalar remainder for columns 4-5
158   for j in 4 .. 6:
159       acc_scalar = 0.0
160       for k in 0 .. 8:
161           acc_scalar += A[i,k] * B[k,j]
162       C[i,j] = acc_scalar

```

### 3.3 Handling Remainder of `linalg.matmul` Transformation

When the column dimension  $N$  is not divisible by the vector length, we must handle the remainder elements. Our pass supports three remainder-handling strategies, each with different performance characteristics.

**3.3.1 Scalar Remainder.** The scalar remainder strategy, already illustrated in the previous subsection, processes remaining elements using a separate scalar cleanup loop. This approach incurs loop-control overhead but avoids masking costs entirely. For small remainders, this overhead is minimal, but it can become significant when the remainder represents a large fraction of the total iterations.

**3.3.2 Masked Remainder.** Masked vectorization handles remainders by using predicate masks to selectively enable or disable vector lanes during computation. Modern SIMD ISAs with first-class predication support, such as AVX-512 and ARM SVE, provide efficient masked load, store, and arithmetic operations [10]. The masked remainder strategy processes the remainder using a single masked vector operation, avoiding the control-flow overhead of scalar loops. However, masking introduces its own costs: predicate setup, masked instruction latency, and potential register pressure from maintaining mask values.

Consider again the  $4 \times 6$  matrix with vector length 4, where we have 2 remainder columns. Using the masked remainder strategy, the transformation generates:

```

184   // Pseudocode (masked remainder)
185   for i in 0 .. 4:
186       // Vectorized path for columns 0-3
187       for j in 0 .. 4 step 4:
188           acc_vec = vector<float,4>(0)
189           for k in 0 .. 8:
190               a_vec = broadcast(A[i,k], 4)
191               b_vec = load_vector(B[k, j : j+3])
192               acc_vec = fma(a_vec, b_vec, acc_vec)
193           store_vector(acc_vec, C[i,j])
194
195   // Masked remainder for columns 4-5

```

```

197     remainder = 2
198     mask = [1,1,0,0]    // enables first 2 lanes
199
200     acc_vec = vector<float,4>(0)
201     for k in 0 .. 8:
202         a_vec = broadcast(A[i,k], 4)
203         b_vec = masked_load(B[k,4], mask)
204         acc_vec = fma(a_vec, b_vec, acc_vec)
205
206     masked_store(acc_vec, C[i,4], mask)

```

3.3.3 *Unrolled Remainder*. The unrolled remainder strategy processes multiple remainder elements within a single loop iteration by unrolling the remainder loop. For small remainders, this approach can reduce loop-control overhead while maintaining scalar computation, avoiding masking costs. The strategy is particularly effective when the remainder size is small and the register pressure from unrolling remains manageable.

When unrolling is applied to the  $4 \times 6$  matrix example, the remainder loop processes both remaining columns (4 and 5) in a single unrolled iteration:

```

214 // Pseudocode (unrolled remainder)
215 for i in 0 .. 4:
216     // Vectorized path for columns 0-3 (same as before)
217
218     // Unrolled remainder: process columns 4 and 5
219     for j in 4 .. 6 step 2:
220         j0 = j
221         j1 = j+1
222
223         acc0 = 0.0
224         acc1 = 0.0
225         for k in 0 .. 8:
226             acc0 += A[i,k] * B[k,j0]
227             acc1 += A[i,k] * B[k,j1]
228
229         C[i,j0] = acc0
230         C[i,j1] = acc1

```

The unrolled approach reduces the number of remainder loop iterations by processing multiple columns per iteration, but requires more registers to hold intermediate results.

### 3.4 Avoiding Remainder using Logical Size

An alternative approach to remainder handling is to avoid remainders entirely by choosing a logical size (LS) that is smaller than the hardware vector size (VS) but divides evenly into the iteration count. When  $N \bmod LS = 0$ , all iterations can use the same masked vector operations consistently, eliminating the need for separate remainder handling code. This strategy is particularly attractive on ISAs with efficient first-class predication support, such as ARM SVE, where masking overhead is minimal.

Consider a  $4 \times 6$  matrix with hardware vector size  $VS = 4$ . Using the masked remainder strategy, we process columns 0–3 with unmasked vector operations, then handle columns 4–5 with a masked remainder. Instead, we can choose a logical size  $LS = 2$  (or  $LS = 3$ ), which divides evenly into  $N = 6$ .

This allows us to process all 6 columns using consistently masked vector operations with stride  $LS$ , avoiding remainder code entirely.

Using logical size  $LS = 2$  for the  $4 \times 6$  matrix example, the transformation generates:

```
// Pseudocode (logical size LS=2)
for i in 0 .. 4:
  for j in 0 .. 6 step 2:
    mask = [1,1,0,0]          // LS=2 active lanes
    acc_vec = vector<float,4>(0)

    for k in 0 .. 8:
      a_vec = broadcast(A[i,k], 4)
      b_vec = masked_load(B[k,j], mask)
      acc_vec = fma(a_vec, b_vec, acc_vec)

    masked_store(acc_vec, C[i,j], mask)

// No remainder: 6 % 2 = 0
```

The key advantage of this approach is uniformity: every iteration uses the same masked vector operations with stride  $LS$ , maintaining consistent vector width throughout. This contrasts with the masked remainder strategy, which uses unmasked operations for the majority of iterations and masks only the final remainder. On ISAs where masking is cheap (e.g., SVE's predicated instructions), the uniform masked approach can outperform mixed unmasked/masked code by avoiding the control-flow divergence and instruction mix heterogeneity.

The logical size strategy requires finding a divisor  $LS$  of  $N$  such that  $LS \leq VS$  and  $N \bmod LS = 0$ . Our pass explores candidate values of  $LS$  (e.g., divisors of  $N$  up to  $VS$ ) and evaluates them using the cost model. When such an  $LS$  exists and the estimated masking overhead is acceptable, this strategy eliminates remainder handling code entirely, resulting in cleaner, more uniform loop structure.

### 3.5 Positioning Remainder

The remainder elements need not always be handled as a tail, they can alternatively be positioned as a head or split between head and tail. This positioning strategy is motivated by the observation that vector address alignment may matter: if the base address is aligned, positioning the remainder at the head might enable better alignment for the main vectorized loop, potentially providing speedup.

We attempted to implement remainder positioning based on base address alignment. However, this approach required loop versioning: maintaining two conditional paths where remainder positioning occurs and where it does not. The overhead from this versioning significantly outweighed any potential alignment benefits, resulting in performance that was much slower than even scalar remainder. Consequently, we removed this strategy entirely from our implementation.

### 3.6 Incorporating All Strategies

Having described three remainder-handling strategies (scalar remainder, masked remainder, unrolled remainder) and the logical size approach to avoid remainders, we can now address how the pass selects the optimal combination. Our pass implements two cost models, analytical and neural-network-based, to evaluate candidate vector shapes and choose the best strategy for each matmul operation.

3.6.1 *Strategy Enumeration.* For a given matrix multiplication with dimensions  $(M, N, K)$  and hardware vector size  $VS$ , the pass enumerates candidate configurations by systematically varying:

- Logical size  $LS \in [1, VS]$ : all possible masking strides that fit within the hardware vector width
- Remainder strategy  $R \in \{\text{masked}, \text{unrolled}\}$ : how to handle leftover elements when  $N \bmod LS \neq 0$

When  $N \bmod LS = 0$  and  $LS = VS$ , the strategy reduces to choosing to max out the vector size. The enumeration generates  $O(VS \times 2)$  candidate configurations per matmul operation, each corresponding to a distinct vector shape  $(LS, R)$ .

3.6.2 *Analytical Cost Model.* Our analytical cost model estimates performance based on ISA characteristics and instruction counting. The model evaluates each candidate configuration by considering the following cost model:

$$\text{Cost} = M \cdot a + N \cdot b$$

where  $M$  is the number of masking events,  $N$  is the number of scalar elements processed via unrolling, and  $a$  and  $b$  are ISA-specific constants calibrated from microbenchmarks.

The constants  $(a, b)$  reflect the relative costs of masking and scalar operations on different ISAs:

- AVX (128-bit):  $a = 0.25$ ,  $b = 0.10$  (masking relatively cheap)
- AVX2 (256-bit):  $a = 1.00$ ,  $b = 0.25$  (masking more expensive)
- AVX-512 (512-bit):  $a = 0.25$ ,  $b = 0.10$  (efficient predication)
- SVE (ARM):  $a = 0.25$ ,  $b = 0.50$  (very cheap masking)

We determine these constants by systematically enumerating candidate  $(a, b)$  values and evaluating each pair against a benchmark suite. Our parameter sweep script tests different  $(a, b)$  combinations by running benchmarks across diverse matrix sizes, comparing the heuristic's selected strategy against the best-performing blind strategy (masked remainder or unrolled remainder) for each test case. The constants that maximize the fraction of tests where the heuristic is within a small tolerance of the best strategy are selected.

The model selects masked remainder when  $\text{Cost}_{\text{mask}} = a < \text{Cost}_{\text{unroll}} = N \cdot b$ , favoring unrolling for small remainders on ISAs where masking is expensive (e.g., AVX2), and preferring masking on ISAs with efficient predication (e.g., AVX-512, SVE).

These constant values reveal interesting architectural patterns. The fact that AVX-512 uses the same  $(a, b)$  as AVX suggests that despite its wider vectors, masking overhead remains relatively low due to efficient predication support. AVX2's higher masking cost ( $a = 1.00$ ) reflects that its 256-bit vectors lack the dedicated mask registers and predication support found in AVX-512, making masked operations more expensive. The SVE values ( $a = 0.25$ ,  $b = 0.50$ ) indicate that while masking is cheap on ARM's scalable vector extensions, unrolling is relatively more expensive, likely due to register pressure constraints or instruction scheduling characteristics.

For logical size selection, the analytical model estimates masking overhead based on whether  $LS = VS$  (no masking) or  $LS < VS$  (uniform masking), then selects the  $LS$  that minimizes total cost while potentially eliminating remainders. When multiple  $LS$  values eliminate remainders, the model prefers larger values to maximize vector width utilization, subject to masking overhead constraints.

3.6.3 *Neural-Network-Based Cost Model.* We initially did not plan to implement a neural-network-based cost model. However, after reading about the deep learning cost model in the Tiramisu compiler [3], we became excited about the idea—since we already had a script to sweep and generate  $(a, b)$  parameters for the analytical model, why not use a neural network as well? This led



us to implement a learned cost model using a feed-forward neural network to capture complex microarchitectural interactions that analytical models might miss.

We train separate neural network models for each ISA (AVX, AVX2, AVX-512, SVE). Each model takes 6 input features: logical size ( $LS$ ), binary indicator for  $LS = VS$ , numeric remainder  $N \bmod LS$ , remainder strategy (masking vs. unrolling), ratio  $LS/N$ , and repetition dimension product. The network uses a dual-branch architecture: a non-linear branch processes strategy features through three fully-connected layers ( $[5] \rightarrow [64] \rightarrow [32] \rightarrow [16]$ ) with ReLU and dropout, while a linear branch handles the repetition count feature directly. The branches are concatenated and passed through final layers to predict speedup.

**3.6.4 Cost Model Integration.** Both cost models are integrated into the pass via command-line flags. The analytical model is used with `--linalg-to-vector-use-heuristic`, while the neural network model is enabled with `--linalg-to-vector-use-model`. When the neural network is enabled, the pass:

- (1) Enumerates all  $(LS, R)$  combinations for the given  $(M, N, K)$  and ISA
- (2) Constructs feature vectors for each candidate
- (3) Calculates costs
- (4) Selects the best candidate given the costs
- (5) Materializes the corresponding vectorized loop structure

The neural network can explore configurations that the analytical model might overlook, such as non-intuitive logical sizes or counter-intuitive strategy choices. However, we find that when the neural network diverges significantly from analytical predictions, its error is often highest, suggesting the analytical model captures the dominant performance factors. We therefore use the analytical model as the primary decision mechanism, with the neural network serving as a complementary approach for validation and exploration.

## 4 Experiment Setup

### 4.1 Hardware Infrastructure

We conducted experiments on AWS EC2 instances to evaluate our vectorization strategies:

- Instance Type: c7i.xlarge
- CPU: 8 vCPUs @ 3.0 GHz base, 3.5GHz Turbo
- Vector Extensions: AVX-512, AVX2, AVX
- Memory: 16GB DDR5
- Cost: \$0.17/hour

We selected the c7i.xlarge instance for its cost-effectiveness while providing access to modern Intel Sapphire Rapids microarchitecture with AVX-512 support. The instance supports three vector width configurations: AVX-512 with 512-bit vectors (16 32bit-floats), AVX2 with 256-bit vectors (8 floats), and AVX with 128-bit vectors (4 floats).

### 4.2 Software Stack

Our experiment is built on LLVM version 17.0.6, with MLIR for intermediate representation and transformation. We compile generated code using Clang17 with `-O3` optimization level targeting x86-64 architecture. For each vector ISA, we apply architecture-specific compilation flags: AVX-512 configurations use `-mavx512f -mavx512vl`, AVX2 uses `-mavx2 -mfma`, and AVX uses `-mavx -mfma`.

Our custom MLIR pass extends the standard `linalg.matmul` dialect with strategy selection capabilities. The pass accepts several command-line flags to control vectorization behavior:



--linalg-to-vector-vector-width specifies the target vector width in bits (128, 256, or 512), while strategy-specific flags including --linalg-to-vector-use-<strategy\_name>, like --linalg-to-vector-use-masked-remainder, and so on. enable different remainder handling approaches.

### 4.3 Benchmark Methodology

We compare five vectorization strategies across three benchmark suites: (1) scalar remainder (baseline), (2) masked remainder, (3) unrolled remainder, (4) heuristic-based selection (analytical cost model), and (5) neural-network-based selection. Our compilation pipeline consists of four stages: (1) our custom MLIR pass vectorizes linalg.matmul operations according to strategy selection, (2) mlir-translate converts MLIR to LLVM IR, (3) llc -O3 generates architecture-specific object files, and (4) linking with a C harness produces the benchmark executable.

**4.3.1 Different-Sized Matrix.** We evaluate strategies using square matrix multiplications with dimensions ranging from  $2 \times 2$  to  $16 \times 16$ . This range captures both perfectly-aligned cases and cases with substantial remainders, allowing us to study performance across different ratios of matrix dimension to vector length.

**4.3.2 HEIR MLP (MNIST).** HEIR’s MLP was the initial benchmark we intended to use, representing a real-world neural network workload. However, we learned along the way that it was not a good benchmark case because all matrix multiplications there have perfectly divisible dimensions and there are only two matmuls, making remainder handling strategy largely irrelevant.

**4.3.3 Custom Transformer Block.** To create a more realistic evaluation, we developed a custom transformer encoder block modeled after BERT-base. This benchmark provides a real-life program workload with multiple matrix configurations, including varied  $K$ -dimension remainders (775, 771, 770, 769, 3075) to exercise different remainder handling scenarios across multiple GEMM operations.

## 5 Experimental Evaluation

### 5.1 Different-Sized Matrix

This benchmark result 1 shows the clearest separation between strategies. On SVE, all non-scalar strategies are 1.45x–1.73x faster than scalar, with masked, heuristic, and model essentially tied ( $\approx 1.72x$ ). On AVX, improvements are modest (1.25x–1.33x) due to blend/mask overhead; heuristic performs best ( $\approx 1.33x$ ). AVX2 demonstrates strong vector-width effects: masked and heuristic are clear winners (2.5x+), with model close behind (2.55x). AVX-512 shows the largest gains: masking and heuristic achieve 6.5x+ faster than scalar, as scalar remainder is particularly slow on AVX-512; the model performs well but is 20% below masked/unrolled strategies. Overall, masking and heuristic dominate on wide-vector ISAs (AVX2/AVX-512), while on SVE all SIMD remainder strategies cluster tightly.

### 5.2 HEIR MLP (MNIST)

This workload consists of just two matmuls with perfectly divisible dimensions, making remainder handling strategy largely irrelevant. This was a surprise for us: we initially expected this real-world workload to provide meaningful remainder handling evaluation, but the perfect alignment eliminated most strategy differences. However, the results still reveal unexpected behavior: On SVE and AVX-512, all strategies achieve  $\approx 1.00x$  as expected for perfectly aligned workloads. On AVX, masked outperforms (1.18x) despite perfect alignment. Most surprisingly, on AVX2 the neural network model shows a strong outlier performance (1.15x faster than scalar), while other strategies cluster around 0.98–1.00x. This unexpected divergence on AVX2—where one would

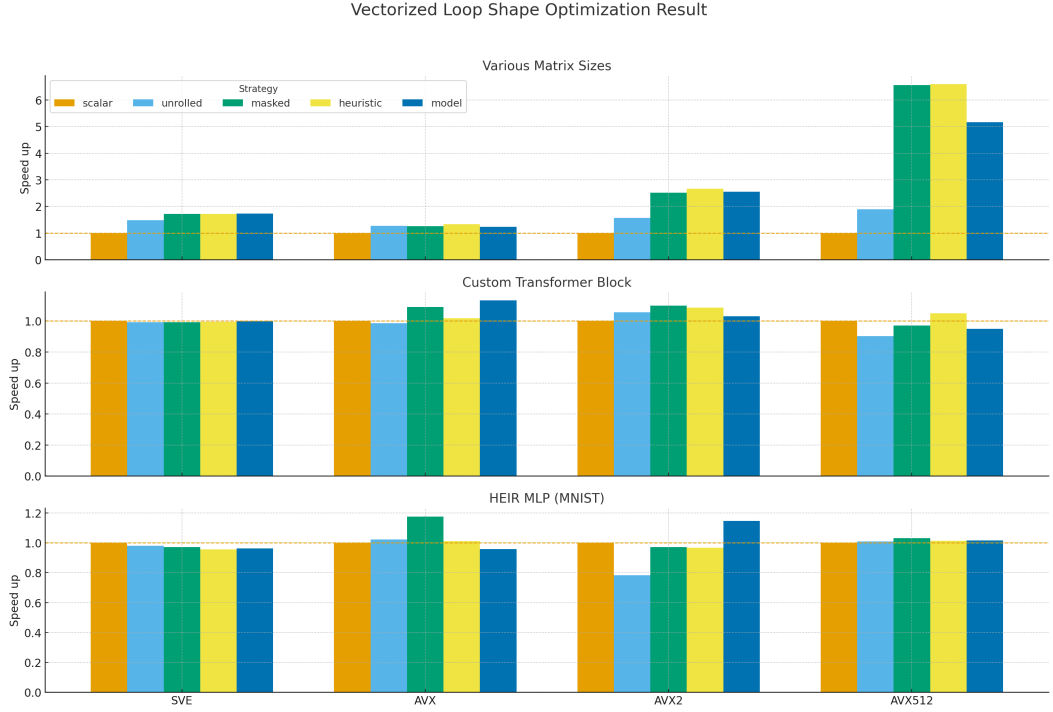


Fig. 1. Final Benchmark Result

expect all strategies to perform identically given perfect alignment—suggests that even in absence of remainder handling, vector shape and instruction selection choices still matter.

### 5.3 Custom Transformer Block

Gains are much smaller here because most products are full-width, diluting remainder effects. On SVE, all strategies achieve  $\approx 1.00x$ , meaning remainder strategy barely matters. On AVX, masked and model outperform scalar ( $1.09x$ - $1.13x$ ), with model performing best ( $1.13x$ ). On AVX2, masked is best ( $1.10x$ ), with heuristic and unrolled close behind; model is slightly below the top. On AVX-512, heuristic is slightly best ( $1.05x$ ), with unrolled/masked/model hovering around  $1.0$ - $1.03x$ . Overall, transformer workloads are remainder-insensitive, showing small but consistent wins from masking/model on AVX/AVX2, while SVE and AVX-512 show minimal differences.

## 6 Surprises and Lessons Learned

**Body Masking Never Materializes.** While our analytical cost model included fully-masked strategy, it never proved optimal in our benchmarks. Interestingly, our neural network model occasionally predicted body masking would win, but empirical measurements consistently showed it underperformed. We expected body masking to perform better on AVX-512 and SVE, where masking overhead is minimal; however, our data contradicts with the hypothesis.

**Simple Heuristics Outperform Neural Network.** Despite training a neural network on thousands of performance samples, our threshold-based heuristic achieved better performance on most iterations. This is most likely because the NN model was not trained on large, diverse dataset, and it wasn't able to pick optimal strategy like heuristic does.

**HEIR Benchmark Anomalies.** The HIER benchmark uses matrix dimensions perfectly divisible by vector width, making remainder handling irrelevant. As expected, we observed little to no difference in performance between strategies, except for AVX2. NN model showed about 15% speedup, while unrolled scalar strategy underperformed.

**Choose benchmark with more caution** Searching for real-world benchmark, we originally anticipated to observe interesting results from HIER MLP benchmark. However, later we realized that HIER benchmark had mostly perfectly divisible dimension, in which remainder strategy doesn't work.

## 7 Conclusion and Future Work

We presented a compiler pass that systematically explores vector shapes and remainder-handling strategies, guided by a cost model. Our evaluation suggests that systematic strategy selection provides substantial benefits on synthetic matrix workloads, with masking and heuristic-based selection achieving up to 6.5x speedup over scalar remainder on wide-vector ISAs (AVX-512). The analytical cost model consistently performs at or near the top across all benchmarks and ISAs, validating principled cost modeling as a strong foundation for automatic vector-shape selection.

However, our results also reveal important limitations: the neural network model, while competitive, occasionally underperforms in challenging cases and shows higher error when diverging from analytical predictions. Real-world workloads like transformer blocks are often remainder-insensitive, showing only modest gains (5–13%) from strategy selection. Some initially promising strategies, such as body masking and remainder positioning, proved ineffective in practice due to overhead or implementation complexity.

Our work demonstrates that vectorization strategy selection is highly ISA and workload-dependent. Efficient predication support (SVE, AVX-512) often makes strategy choice less critical, while architectures without dedicated mask support (AVX2) benefit more from systematic cost modeling. These findings highlight the need for careful, architecture-aware decision-making rather than one-size-fits-all approaches.

Future directions include:

- Extending the pass to support tiled and blocked kernels (e.g. cache-tiling for matrix multiply).
- Training the Neural-network based model with larger data to test its potential of outperforming our analytical model
- Incorporating memory hierarchy models (cache lines, prefetch, NUMA) into the cost model.
- Improving alignment-aware optimizations: while remainder positioning via loop versioning proved too costly, exploring compile-time alignment analysis or profile-guided optimization could enable alignment benefits without runtime overhead.

## 8 Distribution of Total Credit

We distributed our work evenly. (50%-50%)

## References

- [1] 2012. Vectorization: When and How. <https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/vectorization/>. Lecture Notes on SIMD Vectorization.
- [2] Arm Limited. 2023. Using Pragmas to Encourage or Suppress Auto-Vectorization. <https://developer.arm.com/documentation/100891/latest/>. Section "Using pragmas to encourage or suppress auto-vectorization", accessed December 3, 2025.
- [3] Riyadh Baghdadi, Massinissa Merouani, Mohamed-Hicham Leghettas, Kamel Abdous, Taha Arbaoui, Karima Benatchba, and Saman Amarasinghe. 2021. A Deep Learning Based Cost Model for Automatic Code Optimization. *arXiv preprint arXiv:2104.04955* (2021).

- [4] N. Bohm Agostini, J. Haris, P. Gibson, M. Jayaweera, N. Rubin, A. Tumeo, J. Abellán, J. Cano, and D. Kaeli. 2023. AXI4MLIR: User-Driven Automatic Host Code Generation for Custom AXI-Based Accelerators. <https://arxiv.org/abs/2312.14821>.
- [5] Intel Corporation. 2015. Best Practices for SIMD Vectorization. <https://software.intel.com/content/www/us/en/develop/articles/auto-vectorization-in-intel-compilers.html>.
- [6] Ameer Haj-Ali, Nesreen K. Ahmed, Ted Willke, Sophia Shao, Krste Asanovic, and Ion Stoica. 2019. NeuroVectorizer: End-to-End Auto-Vectorization with Deep Reinforcement Learning. *arXiv preprint arXiv:1909.13639* (2019).
- [7] Intel Corporation. 2014. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Technical Report Order Number: 248966-030. Intel Corporation, Santa Clara, CA, USA. <https://www.intel.com/content/www/us/en/content-details/671488/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html> Available online.
- [8] LLVM Project. 2024. *Auto-Vectorization in LLVM*. LLVM Foundation. <https://llvm.org/docs/Vectorizers.html> LLVM 22.0.0git User Guides, accessed December 3, 2025.
- [9] Will Lovett. 2022. LLVM 14 – What’s New and Improved for Arm. <https://developer.arm.com/community/arm-community-blogs/b/tools-software-ides-blog/posts/llvm-14>. Arm Community Blog, accessed December 3, 2025.
- [10] Andrzej Warzynski and Diego Caballero. 2023. RFC: Scalable Vectorisation in Linalg. <https://discourse.llvm.org/t/rfc-scalable-vectorisation-in-linalg/70419>. LLVM Discourse, accessed December 3, 2025.